PList
Listas y Recetas frescas

Manual completo del proyecto PList

Listas y Recetas frescas

Guía para entender, mantener, probar y seguir desarrollando la aplicación.

Dato	Información
Propietario del proyecto	Periche / Wasadingo
Repositorio	https://github.com/Wasadingo/plist-app
Aplicación publicada	https://wasadingo.github.io/plist-app/
Backend	Supabase · proyecto qfavfkxgszapbiquafrw
Correo principal de administración	periche1995@gmail.com
Estado del documento	Manual vivo: actualizarlo después de cada cambio importante

Importante. Este manual no incluye contraseñas, claves service_role, claves privadas VAPID ni API keys secretas. Esas credenciales se guardan solo en Supabase y nunca deben publicarse en GitHub.

Cómo usar este manual

Este documento está pensado para una persona que no trabaja todos los días con programación. Primero explica qué es PList con palabras sencillas. Después describe cada parte técnica, dónde está cada archivo y qué pasos hay que seguir para cambiar imágenes, correos, usuarios, listas, permisos o notificaciones.

Cuando aparezca la palabra usuario se refiere a la persona que utiliza PList. Cuando aparezca administrador se refiere a la cuenta con permisos globales. Cuando aparezca Supabase se refiere al servicio que guarda los datos, autentica las cuentas y ejecuta funciones del servidor.

Índice práctico

- 1. Qué es PList y qué problema resuelve
- 2. Arquitectura explicada de forma sencilla
- 3. Servicios y cuentas utilizadas
- 4. Estructura de carpetas y archivos
- 5. Funciones principales de la aplicación
- 6. Usuarios, perfiles y administrador
- 7. Listas compartidas y permisos
- 8. Notificaciones y Web Push
- 9. Correos, Brevo y feedback
- 10. PWA y compatibilidad con dispositivos
- 11. Cómo hacer cambios habituales
- 12. Pruebas y solución de problemas
- 13. Historial de cambios
- 14. Lista de comprobación antes de publicar

1 Qué es PList y qué problema resuelve

PList es una aplicación web para organizar la compra y guardar recetas. Permite que una persona cree listas privadas, comparta una lista con otro usuario y trabaje con ella de forma colaborativa. También permite guardar recetas, recibir invitaciones, consultar actividad y recibir avisos cuando otro miembro modifica una lista.

La aplicación se diseñó para funcionar desde ordenador, portátil, tablet y móvil. También puede instalarse como PWA, es decir, como una aplicación que se abre desde un icono y puede enviar notificaciones del dispositivo.

Necesidad	Respuesta de PList
Organizar compras	Listas personales con productos, cantidades y estado comprado.
Trabajar con otra persona	Listas compartidas, invitaciones y rol de colaborador.
No perder cambios	Supabase guarda los datos en la nube y el navegador mantiene la sesión.
Saber qué ha pasado	Actividad reciente y notificaciones internas.
Recibir avisos	Notificaciones del navegador y Web Push por dispositivo.
Gestionar la aplicación	Perfil administrador protegido con PIN de cuatro dígitos.

2 Arquitectura explicada de forma sencilla

PList tiene tres capas. La primera es la pantalla que ve la persona. La segunda es Supabase, que guarda los datos y controla los permisos. La tercera son las funciones que realizan tareas privadas, como enviar correos o avisos push.

Qué ocurre cuando una persona utiliza PList

1. La persona abre la dirección de GitHub Pages.
2. El navegador carga HTML, estilos, imágenes y JavaScript.
3. El JavaScript comprueba si hay una sesión de Supabase.

4. Si no hay sesión, aparece la pantalla de entrada o creación de cuenta.
5. Si hay sesión, se carga el perfil real y solo se consultan los datos de ese usuario.
6. Cuando se cambia una lista, Supabase guarda el cambio y crea las notificaciones necesarias.
7. Si el dispositivo tiene Web Push activado, una Edge Function envía el aviso aunque la aplicación esté cerrada.

Parte	Función	Ejemplo
Frontend	Pantallas e interacción	app.js, auth.js, styles.css
Base de datos	Usuarios, listas, productos y permisos	Supabase PostgreSQL
Autenticación	Registro, entrada y recuperación de contraseña	Supabase Auth
Funciones servidor	Correo, administración y push	Edge Functions
Publicación	Hace visible la aplicación	GitHub Pages y GitHub Actions
Instalación	Convierte la web en PWA	manifest.webmanifest y sw.js

Vista general

```
Usuario
  └── PList publicada en GitHub Pages
        ├── Supabase Auth: identifica la cuenta
        ├── Supabase Database: guarda listas y recetas
        ├── Supabase Realtime: avisa de cambios en directo
        ├── Edge Functions: correo, PIN y Web Push
        └── Service Worker: PWA y avisos del dispositivo
```

3 Servicios y cuentas utilizadas

Servicio	Para qué se utiliza	Qué no debe hacerse
GitHub	Repositorio del código y publicación automática.	No guardar claves privadas ni contraseñas.
GitHub Pages	Dirección pública de la aplicación.	No editar manualmente archivos publicados.
Supabase	Base de datos, usuarios, permisos, funciones y Realtime.	No exponer service_role en el navegador.
Brevo	Correo de confirmación, recuperación y feedback.	No usar la cuenta SMTP como remitente.
Navegador / PWA	Permisos y recepción Web Push.	No cerrar sesión si se quieren recibir avisos.

Direcciones principales. Supabase: https://qfavfkxgszapbiquafrw.supabase.co · GitHub: https://github.com/Wasadingo/plist-app · Aplicación: https://wasadingo.github.io/plist-app/

Las claves públicas pueden aparecer en el frontend porque están protegidas por RLS. Las claves secretas se guardan en Supabase Secrets. Esta separación es importante para que una persona que visite la web no pueda administrar la base de datos.

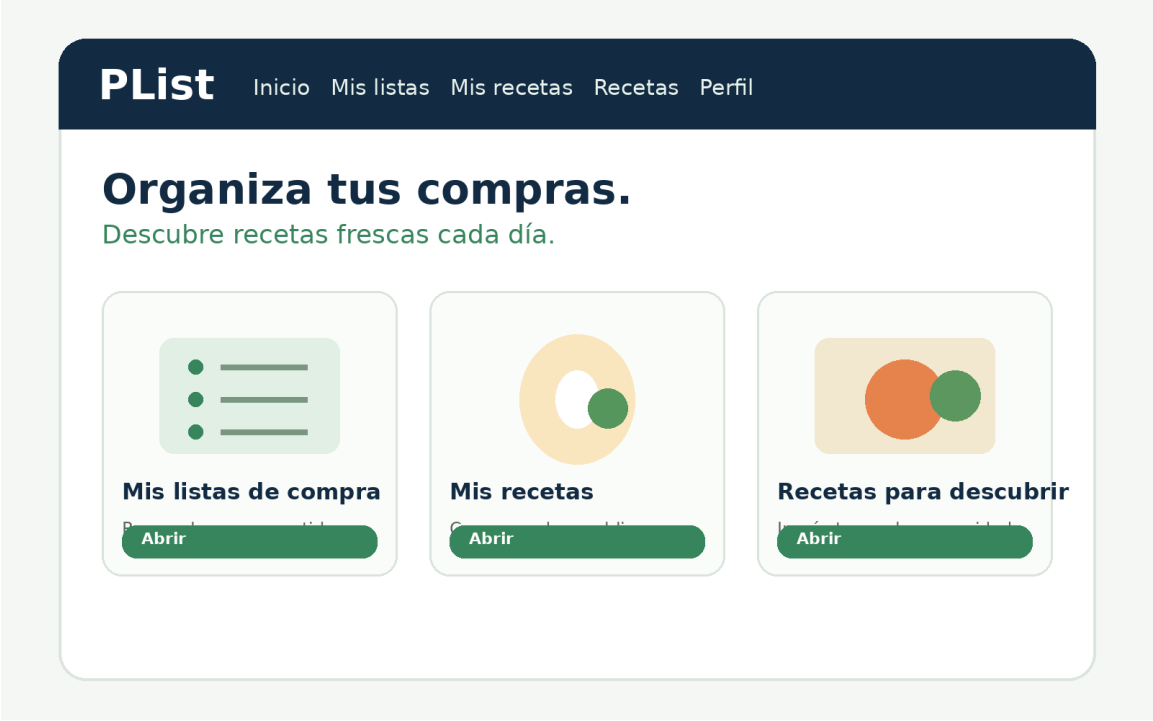
4 Estructura de carpetas y archivos

Ruta	Qué contiene	Cuándo tocarla
index.html	Estructura general, navegación, cabecera, menús y carga de scripts.	Al cambiar el esqueleto principal o añadir un recurso.
styles.css	Colores, tamaños, tarjetas, formularios, responsive y PWA visual.	Al cambiar apariencia o compatibilidad de pantallas.
app.js	Estado de usuario, inicio, listas, recetas, actividad, menús y acciones.	Al cambiar comportamiento de las pantallas autenticadas.
auth.js	Crear cuenta, entrar, recuperar contraseña y confirmar	Al cambiar el acceso o los textos

	correo.	de autenticación.
supabase-client.js	Conexión del navegador con Supabase y funciones disponibles.	Al añadir una operación de base de datos.
db.js	Persistencia local del navegador para preferencias y actividad local.	Al cambiar IndexedDB o datos locales.
supabase-config.public.js	URL, clave pública y clave pública VAPID.	Al cambiar de proyecto Supabase o de pareja VAPID pública.
sw.js	Caché PWA, recepción push y apertura al pulsar una notificación.	Al cambiar PWA, caché o enlaces de notificaciones.
manifest.webmanifest	Nombre, iconos, color, modo instalado y URL inicial.	Al cambiar instalación o identidad visual.
assets/	Logotipos e imágenes visuales.	Al cambiar logotipos o recursos.
supabase/schema.sql	Esquema inicial de tablas y políticas.	Solo para revisar o reconstruir una base nueva.
supabase/migrations/	Cambios incrementales del backend.	Cada cambio de base de datos debe tener una migración.
supabase/functions/	Código servidor que no debe ejecutarse en el navegador.	Correo, push y otras tareas privadas.
.github/workflows/	Automatización de publicación en GitHub Pages.	Solo al cambiar el proceso de despliegue.

Regla sencilla para elegir archivo

Si el cambio es visual, normalmente se toca styles.css, index.html o assets. Si cambia una pantalla o un botón, se toca app.js o auth.js. Si cambia dónde se guardan los datos o quién puede verlos, se crea una migración SQL. Si utiliza una clave privada o llama a Brevo/Web Push, debe ir en una Edge Function.



Referencia visual de la interfaz inicial de PList

5 Funciones principales de la aplicación

Acceso y cuenta

- Crear cuenta con nombre visible, apodo, correo y contraseña.
- Confirmar el correo mediante Supabase Auth y Brevo.
- Entrar y salir de la cuenta.
- Solicitar un enlace para recuperar la contraseña.
- Mostrar siempre el perfil real de la cuenta autenticada.

Listas de compra

- Crear lista personal o compartida.
- Añadir productos y cantidades.
- Marcar un producto como comprado o pendiente.
- Editar o eliminar una lista mediante el menú de tres puntos.
- Abrir una lista para consultar su progreso.

Recetas

- Consultar recetas disponibles.
- Guardar recetas en favoritos.
- Crear, editar y eliminar recetas desde el menú de acciones.

Ayuda y feedback

El formulario de Ayuda y feedback permite enviar el correo de contacto, el tipo de mensaje y una explicación. La información se guarda en la tabla feedback y se envía a periche1995@gmail.com mediante la función send-feedback y Brevo.

6 Usuarios perfiles y administrador

Cada cuenta tiene un perfil normal. La cuenta autorizada para administrar PList puede configurar un PIN privado de cuatro dígitos y cambiar entre perfil normal y perfil administrador.

Perfil	Puede hacer	No debe hacer
Usuario normal	Gestionar sus listas, sus productos, sus recetas y las listas compartidas permitidas.	Ver o modificar datos de otras cuentas sin invitación.
Colaborador	Añadir productos, marcarlos como comprados y modificar o eliminar los que ha creado, según las reglas de la lista.	Eliminar productos creados por otra persona si no tiene permiso.
Propietario	Compartir la lista, administrar miembros y gestionar el contenido de su lista.	Compartir el PIN o las claves del sistema.
Administrador	Acceder a funciones globales autorizadas mediante PIN.	Usar el PIN como sustituto de la contraseña o compartirlo.

Seguridad. El PIN no debe aparecer en el código ni en GitHub. Se guarda como hash y se comprueba mediante funciones de Supabase.

7 Listas compartidas y permisos

El propietario busca al destinatario mediante su apodo público. Supabase crea una invitación en list_invitations y una notificación en notifications. El destinatario puede aceptar o rechazar desde su panel.

```
Propietario
  └── crea lista compartida
```

```
└── invita mediante apodo
      └── destinatario recibe notificación
            ├── aceptar → list_members con rol collaborator
            └── rechazar → invitación cerrada sin acceso
```

Tabla o función	Responsabilidad
shopping_lists	Nombre, propietario y tipo de lista.
shopping_items	Productos, cantidades, estado comprado y autor.
list_members	Miembros y rol owner o collaborator.
list_invitations	Invitaciones pendientes, aceptadas o rechazadas.
notifications	Avisos internos de invitaciones y cambios.
create_list_invitation	Comprueba propietario y crea una invitación.
respond_list_invitation	Acepta o rechaza, añade miembro y cierra el aviso.

8 Notificaciones y Web Push

Hay dos mecanismos relacionados pero diferentes. Supabase Realtime actualiza la aplicación cuando está abierta. Web Push envía un aviso al dispositivo cuando la PWA está cerrada, siempre que la persona haya concedido permiso y no haya cerrado sesión.

Situación	Resultado
Aplicación abierta	Se actualiza el icono y aparece la notificación interna.
Aplicación en segundo plano	Puede aparecer el aviso del navegador si la suscripción está activa.
PWA cerrada	La Edge Function envía Web Push al dispositivo registrado.
Sesión cerrada	Se elimina la suscripción del dispositivo para esa cuenta.
iPhone	Hay que instalar PList como PWA desde Safari y usar iOS/iPadOS compatible.

Piezas necesarias

- push_subscriptions: guarda una suscripción por cuenta y dispositivo.
- Service worker: recibe el push y abre la ruta correcta de GitHub Pages.
- send-push: busca las suscripciones del destinatario y envía el aviso.
- Trigger o webhook: llama a send-push cuando entra una fila nueva en notifications.
- Claves VAPID: permiten demostrar que el envío pertenece a PList.

Problema típico. Si llega el aviso pero al pulsarlo aparece 404, revisar la ruta del service worker y comprobar que contiene /plist-app/. La corrección actual usa self.registration.scope.

9 Correos Brevo y feedback

Brevo se utiliza para enviar mensajes de correo. El remitente debe ser un correo verificado en Brevo. El usuario SMTP de Brevo no debe utilizarse como dirección From.

Dato	Uso	Valor o ubicación
FEEDBACK_RECIPIENT_EMAIL	Destino de sugerencias	periche1995@gmail.com
FEEDBACK_SENDER_EMAIL	Remitente verificado	Correo verificado dentro de Brevo
BREVO_API_KEY	Permite enviar correo transaccional	Secreto en Supabase
IP autorizada	Seguridad opcional de Brevo	Puede variar en funciones serverless

Error 401 de Brevo. Si aparece un mensaje de IP no reconocida, revisar Security / Authorised IPs en Brevo. Para una Edge Function suele ser más estable una clave exclusiva con permiso de correo transaccional y sin restricción de IP fija.

10 PWA y compatibilidad

Dispositivo	Qué hacer
Ordenador	Abrir Chrome o Edge, permitir avisos y, si se desea, instalar PList.
Android	Abrir con Chrome, instalar la PWA y activar avisos desde Perfil y configuración.
iPhone o iPad	Abrir con Safari, añadir a pantalla de inicio, abrir desde el icono y aceptar avisos.
Tablet	Usar el navegador compatible y activar el permiso de notificaciones.

El archivo manifest.webmanifest define el nombre y la instalación. El service worker mantiene recursos de la aplicación en caché, recibe push y abre la aplicación al pulsar la notificación. Cada cambio importante del service worker debe aumentar el nombre de caché, por ejemplo plist-shell-v7 a plist-shell-v8.

11 Cómo hacer cambios habituales

Cambiar el logotipo

8. Sustituir el archivo correspondiente dentro de assets, manteniendo el nombre si no se quiere cambiar el código.
9. Comprobar que el formato es PNG y que tiene fondo transparente cuando se use en la pantalla de acceso.
10. Revisar index.html, auth.js y manifest.webmanifest si se cambia el nombre del archivo.
11. Aumentar la caché del service worker si el navegador conserva la imagen antigua.

Cambiar textos

- Textos de entrada y recuperación: auth.js.
- Textos de inicio, listas, recetas y perfil: app.js.
- Textos generales de navegación: index.html.

Cambiar colores o tamaños

- Variables y colores principales: styles.css.
- Diseño móvil: bloque @media de styles.css.
- Colores de marca de documentos y manuales: no afectan a la aplicación.

Cambiar el correo receptor de feedback

12. Entrar en Supabase → Edge Functions → send-feedback → Secrets.
13. Cambiar FEEDBACK_RECIPIENT_EMAIL.
14. No modificar el código público ni subir la API key a GitHub.
15. Probar el formulario desde Ayuda y feedback.

Añadir una tabla o campo nuevo

16. Crear una nueva migración dentro de supabase/migrations/.
17. Activar RLS si la tabla contiene datos de usuarios.
18. Crear políticas para select, insert, update y delete según el rol.
19. Añadir la operación correspondiente en supabase-client.js.
20. Conectar la operación a app.js y probar con dos cuentas.

12 Pruebas y solución de problemas

Síntoma	Causa habitual	Comprobación
Aparece Alex o contenido de ejemplo	Caché antigua o datos demo.	Abrir la versión con ?v=commit, revisar service worker y limpiar caché PWA.

No llega correo	SMTP/Brevo o remitente no verificado.	Revisar Secrets y logs de la Edge Function.
Brevo 401	IP no autorizada o API key limitada.	Revisar Authorised IPs y permiso de correo transaccional.
No llega push cerrado	No hay fila en push_subscriptions o falla send-push.	Revisar tabla, webhook y logs de send-push.
Push abre 404	Ruta incorrecta en service worker.	Usar self.registration.scope y renovar caché.
Perfil muestra otra persona	Sesión/caché antigua o consulta sin filtro.	Cerrar sesión, comprobar user_id y revisar listLists.
Invitación sigue pendiente	No se ejecutó migración 007.	Revisar respond_list_invitation y read_at.

Prueba mínima para una publicación

- Crear una cuenta nueva y confirmar el correo.
- Entrar desde un segundo navegador o móvil.
- Crear una lista personal y añadir un producto.
- Crear una lista compartida e invitar a otra cuenta.
- Aceptar la invitación y comprobar el rol colaborador.
- Añadir y completar productos desde ambas cuentas.
- Limpiar notificaciones.
- Cerrar completamente la PWA y probar Web Push.
- Cerrar sesión y confirmar que el dispositivo deja de recibir avisos.

13 Historial de cambios

Commit	Cambio
5c839c0	Corrección del enlace de apertura de notificaciones push.
5464f1b	Web Push para avisos con la PWA cerrada.
655a545	Limpieza de notificaciones y cierre de invitaciones.
df6d32f	Avisos en tiempo real para listas compartidas.
7854ffd	Eliminación de datos de demostración del frontend.
42ecfa1	Ayuda, feedback y envío mediante Brevo.
6b46933	Cambio explícito entre perfil normal y administrador.
7ab6693	Corrección de pgcrypto para el PIN.
42f48d0	Recuperación de contraseña.
a31a515	Invitaciones y notificaciones conectadas con Supabase.
38bc4a5	Restricciones para productos de colaboradores.

14 Lista de comprobación antes de publicar

- ☐ El código no contiene nombres de usuarios de prueba ni datos del prototipo.
- ☐ No hay claves secretas dentro de archivos públicos ni commits.
- ☐ La aplicación carga el perfil del usuario autenticado.
- ☐ Las migraciones SQL están ejecutadas en Supabase.
- ☐ Las políticas RLS protegen las tablas de usuarios.
- ☐ La función send-feedback tiene sus secretos configurados.
- ☐ La función send-push tiene sus secretos VAPID configurados.
- ☐ El webhook o trigger de notifications funciona.

- ☐El workflow de GitHub Pages aparece en verde.
- ☐La PWA abre la ruta /plist-app/ y no devuelve 404.
- ☐Se ha probado la aplicación con dos cuentas diferentes.
- ☐Se ha probado al menos un ordenador y un móvil.
- ☐Se ha comprobado la limpieza de notificaciones y el cierre de sesión.

Apéndice A Datos que conviene guardar aparte

Para proteger el proyecto, conviene guardar estos datos en un gestor de contraseñas o documento privado separado del repositorio: contraseña de GitHub, acceso de Supabase, API key de Brevo, service_role de Supabase, PIN de administrador, VAPID private key y PUSH_WEBHOOK_SECRET. Este manual solo explica el nombre y la ubicación de cada dato, no almacena sus valores privados.

Apéndice B Regla de mantenimiento

Cada vez que se haga un cambio importante, actualizar este manual con la fecha, el motivo, el archivo o migración afectada y la prueba realizada. Si el cambio modifica la base de datos, crear una migración nueva; no editar una migración antigua que ya se haya ejecutado en producción.